CoT 与 ReAct — 学习笔记

> **说明**：在当前工作区全文检索中，**未发现**你此前单独保存的「CoT / ReAct」旧稿文件名；`学习资料-AI` 目录下已有 **`agentic-design-patterns-zh-20251208.pdf`**（Agent 设计模式，常含 ReAct 等）。本文档为**补全整理版**，便于与《AI辅助编程范式与工程实践》对照复习。

一、术语对照

英文/缩写	中文常称	本质
CoT	Chain-of-Thought，思维链	让模型**显式写出中间推理步骤**，再给出结论，减少跳步错误
ReAct	Reasoning + Acting	**交错**：想一步（Thought）→ 用行动/工具（Action）→ 看结果（Observation）→ 再想…… 直到结束

易混：**ReAct** 里的「Act」是 **Agent 对外部工具的动作**（查库、搜索、调 API），不是前端框架 **React**。

二、CoT（思维链）

2.1 要解决什么问题

- 复杂算术、逻辑、多约束问题时，模型若**直接吐答案**，容易错在**隐性跳步**。
- **CoT** 通过「先分步推理」提高可检查性；部分场景也提升准确率（因模型被迫放慢）。

2.2 典型写法（示意）

```
请一步一步推理，再给出最终答案：
问题：……
要求：每步用简短句子说明依据，最后单独一行写「结论：」。
```

2.3 好处与代价

好处	代价
便于人审模型错在哪一步	输出更长，**延迟与费用**上升
可截取中间步骤做审计	仍可能「一本正经胡说」——需事实与工具约束

2.4 与「编程范式」的对应（你笔记里的层）

- 偏 **Spec**：你规定了**输出结构**（先推理后结论）。
- 若配合 **禁止编造**：需 **Harness**（事实必须来自工具/数据）+ **Context**（给足材料）。

三、ReAct（推理 + 行动）

3.1 要解决什么问题

- 仅靠**纯文本推理**无法获取**实时信息**（股价、天气、私有库）。
- ReAct 把 **推理** 和 **工具调用** 绑成循环：**想** → **做** → **看** → **再想**。

3.2 循环结构（概念）

```
Thought：当前子目标是什么，为什么。
Action：调用某个工具（如 search、read_file、run_python）。
Observation：工具返回什么。
（重复直到）Thought：可以给出最终答案。
Answer：……
```

3.3 好处与代价

好处	代价
能接真实世界与私有数据	工具多时代价高、易**死循环或胡调工具**
轨迹可记录，利于调试 Agent	需要 **Harness**（权限、超时、重试上限）

3.4 与 Cursor / 编码的关系

- Cursor Agent **多步改代码、跑终端、读文件**，本质接近 **ReAct 式循环**；你仓库里的 **Skills**、**Rules** 就是在做 **Harness + Context**。
- **不是**「用了 ReAct 这个词才算」——看的是**是否有 Thought–Action–Observation 闭环**。

四、CoT vs ReAct（对比表）

维度	CoT	ReAct
是否必须工具	否	是（否则退化为纯 CoT）
典型输出	文字推理链	推理 + 工具调用轨迹
主要风险	幻觉推理	工具误用、成本、循环
面试一句话	「让模型把思考写出来」	「推理与行动交错直到任务完成」

五、面试可能怎么问（简答）

Q：CoT 和 Few-shot 有什么区别？
A：Few-shot 是给例子；CoT 强调**中间推理步骤**的展示形式，可和 few-shot 一起用（例子里也写步骤）。

Q：ReAct 和纯函数调用 Agent 有何不同？
A：ReAct 强调**显式 Thought** 与 **Observation 反馈**交替；纯工具链可能不展示推理，只调工具。

Q：为什么生产环境要给 Agent 加 Harness？
A：限制可调工具、路径与次数，防止越权与死循环；ReAct 越自由，越要缰绳。

Q：思维链会泄露隐私吗？
A：若把敏感数据写进「推理过程」日志，可能泄露；应对日志脱敏、对 Thought 做策略控制。

六、与本目录其它资料的关系

文件	关系
`agentic-design-patterns-zh-20251208.pdf`	建议通读；多涵盖 **Agent 模式**（含 ReAct 类思想）
`AI辅助编程范式与工程实践_学习笔记.md`	**Vibe / Spec / Context / Harness / Agentic** 与本文 **CoT / ReAct** 互补：前者偏**工程化用法**，后者偏**推理与行动机制**

七、一页复习

- **CoT**：先想清步骤，再答——治**跳步**。
- **ReAct**：想一步、动一步、看一步——治**闭门造车**。
- 真上线：**Harness + Context** 缺一不可。

整理日期随仓库更新；技术名词以论文与主流框架表述为准。